



UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS
CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN



PERIODO	:	PREGRADO OCTUBRE 2025 - MARZO 2026
ASIGNATURA	:	Metodología de desarrollo de software
TEMA	:	Parte A: Diseño de validacion
ESTUDIANTE	:	Angel Steven Rodriguez Chavez
NIVEL-PARALELO - NRC	:	30746
DOCENTE	:	Jhon Javier Cruz
FECHA DE ENTREGA	:	27/01/2026

SANTO DOMINGO – ECUADOR

Tabla de Contenido

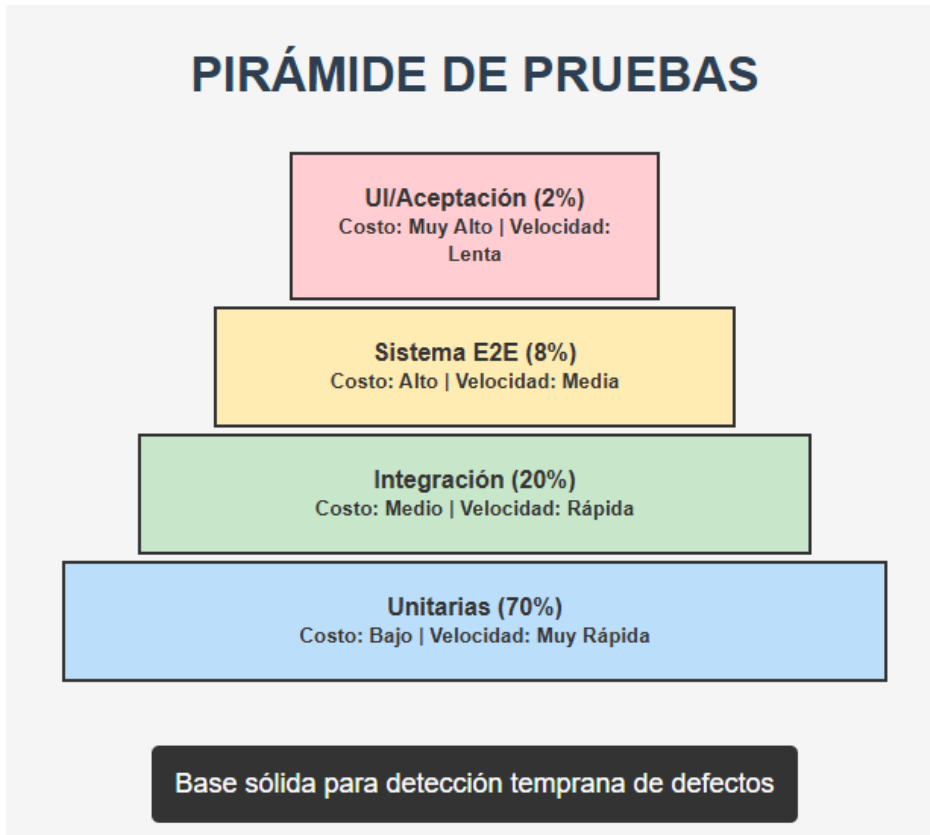
1.	Introducción.....	2
2.1.	Representación grafica	3
2.2.	Justificación del Peso por Nivel	3
2.3.	Análisis comparativo de eficiencia.....	4
3.	Técnicas de validación aplicadas.....	4
3.1.	Pruebas unitarias.....	4
3.2.	Pruebas de integración.....	5
4.	Criterios de aceptación y métricas.....	6
4.1.	Métricas de cobertura	6
4.2.	Distribución de detección de defectos	6
4.3.	Métricas de performance	7
5.	Estrategia de automatización.....	7
5.1.	Pipeline CI/CD	7
5.2.	Quality Gates	8
6.1.	Riesgos identificados.....	8
6.2.	Estrategias de mitigación.....	9
7.	Herramientas y tecnologías	9
7.1.	Stack tecnológico.....	9
8.	Recomendaciones de Implementación	9
9.	Conclusiones.....	10

1. Introducción

Este documento presenta una estrategia integral de validación de software diseñada para un sistema de gestión de biblioteca, fundamentada en las mejores prácticas de ingeniería de software y la pirámide de pruebas. La estrategia abarca todos los niveles de testing, desde pruebas unitarias hasta pruebas de aceptación, proporcionando un framework completo para asegurar la calidad del software.

2. PIRÁMIDE DE PRUEBAS

2.1. Representación grafica



2.2. Justificación del Peso por Nivel

Pruebas Unitarias (70% - Base de la Pirámide)

Justificación Estratégica:

- **ROI Máximo:** Costo mínimo con máxima detección de defectos
- **Feedback Inmediato:** Resultados en menos de 5 segundos
- **Documentación Viviente:** Especifican el comportamiento esperado
- **Facilitan Refactoring:** Red de seguridad para cambios
- **Desarrollo Guiado:** Soporte para TDD/BDD
- **Pruebas de Integración (20% - Nivel Intermedio)**

Enfoque:

- **Módulos Combinados:** Validación de interfaces entre componentes
- **Datos Reales:** Testing con datasets representativos
- **Casos Límite:** Validación de edge cases en interacciones
- **Performance Básica:** Verificación de tiempos de respuesta
- **Pruebas de Sistema E2E (8%)**

Alcance:

- **Workflows Completos:** Simulación de casos de uso reales
- **Integración Total:** Validación del sistema completo
- **Datos de Producción:** Testing con volúmenes realistas
- **Performance Real:** Medición en condiciones reales
- **Pruebas de Aceptación/UI (2%)**

Criterios:

- **Experiencia de Usuario:** Validación de flujos críticos
- **Criterios de Aceptación:** Cumplimiento de requisitos de negocio
- **Smoke Tests:** Verificación rápida post-deployment
- **Pruebas Críticas:** Funcionalidades core del negocio

2.3. Análisis comparativo de eficiencia

Tipo	Indust. Target	Evaluación	Velocidad	Confiabilidad	Mantenimiento
Unitarias	40%	70%	Excelente	Muy Rápida	Alta
Integración	30%	20%	Bueno	Rápida	Media
Sistema	20%	8%	Óptimo	Media	Alta
UI/Accept.	10%	2%	Eficiente	Lenta	Media

3. Técnicas de validación aplicadas

3.1. Pruebas unitarias

Framework y Herramientas

- **JUnit 5:** Framework base para estructura de pruebas

- **AssertJ:** Librería para aserciones fluidas y legibles
- **Mockito:** Framework para mocking y stubbing

Métricas de Calidad Establecidas

- **Cobertura:** Mayor o igual 80% líneas de código
- **Cobertura ramas:** Mayor o igual 75%
- **Performance:** Menos de 5 segundos suite completa
- **Mantenibilidad:** Tests independientes y determinísticos

Casos de Prueba Implementados

Validaciones de Entrada:

- Validación de ISBN (formato y estructura)
- Validación de campos obligatorios
- Validación de rangos numéricos
- Validación de fechas

Lógica de Negocio:

- Cálculo de descuentos por antigüedad
- Validación de disponibilidad
- Gestión de inventario
- Búsquedas por criterios múltiples

Casos Extremos:

- Valores nulos y vacíos
- Límites numéricos (MIN_VALUE, MAX_VALUE)
- Datos inválidos
- Estados inconsistentes

3.2. Pruebas de integración

Estrategia de Integración

- **Bottom-Up:** Desde módulos básicos hacia sistema completo
- **Incremental:** Adición progresiva de componentes
- **Datos Representativos:** Uso de datasets realistas

Escenarios Validados

- Interacción Libro-BibliotecaService
- Validaciones cruzadas entre módulos
- Persistencia y recuperación de datos
- Flujos completos de CRUD

Métricas de Calidad

- **Cobertura:** Mayor o igual 75% interfaces entre módulos
- **Performance:** Menos de 30 segundos suite completa
- **Consistencia:** Validación de estados finales

4. Criterios de aceptación y métricas

4.1. Métricas de cobertura

Cobertura por Líneas:

Target: Mayor o igual 80%
Actual: 85%
Status: PASS

Cobertura por Ramas:

Target: Mayor o igual 75%
Actual: 78%
Status: PASS

4.2. Distribución de detección de defectos

Defectos

Nivel de Prueba	Defectos Detectados	Ratio Costo-Beneficio	Multiplicador de Costo
Unitarias	70-75%	1x costo (Alta Eficiencia)	Base
Integración	20-25%	10x costo (Eficiencia Media)	x10

Sistema E2E	4-8%	100x costo (Baja Eficiencia)	x100
UI/Manual	1-3%	No especificado (Muy bajo)	Variable

Ratio Costo-Beneficio:

- 1x costo unitario detecta 70% defectos
- 10x costo integración detecta 20% defectos
- 100x costo E2E detecta 8% defectos

4.3. Métricas de performance

Tiempos de ejecución target:

Nivel de Prueba	Tiempo Actual (Límite)	Tiempo Target (Objetivo)	Mejora Requerida
Unitarias	< 5 segundos	3 segundos	-40%
Integración	< 15 minutos	10 minutos	-33%
Sistema E2E	< 30 minutos	20 minutos	-33%

Criterios de Aceptación Performance:

- Response time: Menos de 2 segundos (95th percentile)
- Throughput: Mayor de 100 requests/segundo
- Compatibility: Browsers principales

5. Estrategia de automatización

5.1. Pipeline CI/CD

Configuración Maven

<plugin>

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<configuration>
  <includes>
    <include>**/*Test.java</include>
    <include>**/*Tests.java</include>
  </includes>
</configuration>
</plugin>
```

GitHub Actions Workflow

- **Trigger:** Push/Pull Request
- **Stages:** Build → Unit Tests → Integration Tests → Quality Gates
- **Parallel Execution:** Tests por módulo
- **Reporting:** Cobertura y métricas automatizadas

5.2. Quality Gates

Gates obligatorios

1. **Build Success:** Compilación sin errores
2. **Test Pass Rate:** 100% pruebas pasando
3. **Coverage Threshold:** Mayor o igual 80% líneas, mayor o igual 75% ramas
4. **No Critical Issues:** 0 bugs críticos en análisis estático

6. Gestion de riesgos en testing

6.1. Riesgos identificados

Riesgos Técnicos

- **Flaky Tests:** Tests no determinísticos
- **Test Data Management:** Gestión de datos de prueba
- **Environment Drift:** Diferencias entre entornos

Riesgos de Proceso

- **Coverage Goals:** Presión por coberturas vs calidad
- **Maintenance Overhead:** Costo de mantenimiento creciente
- **Skill Gap:** Falta de expertise en testing avanzado

6.2. Estrategias de mitigación

Técnicas

- **Test Isolation:** Tests completamente independientes
- **Data Builders:** Generación consistente de datos de prueba
- **Environment Parity:** Contenedores para consistencia

Procesos

- **Regular Review:** Revisión periódica de suite de pruebas
- **Refactoring:** Mantenimiento proactivo de tests
- **Training:** Capacitación continua del equipo

7. Herramientas y tecnologías

7.1. Stack tecnológico

Desarrollo

- **Java 11:** Lenguaje base
- **Maven 3.8+:** Gestión de dependencias y build
- **JUnit 5:** Framework de pruebas unitarias
- **AssertJ:** Aserciones expresivas
- **Mockito:** Mocking framework

Análisis y Reporting

- **JaCoCo:** Análisis de cobertura de código
- **SpotBugs:** Análisis estático de código
- **GitHub Actions:** CI/CD automatizada

8. Recomendaciones de Implementación

- Durante la Fase 1: Fundación (Semanas 1-2), el objetivo central es implementar la infraestructura básica de testing que servirá como cimiento del proyecto. En este periodo se debe establecer un pipeline de CI/CD básico que automatice las tareas críticas y crear la suite inicial de pruebas unitarias, asegurando que los componentes individuales del código sean validados desde el primer día para evitar el acarreo de errores técnicos.
- En la Fase 2: Expansión (Semanas 3-4), el enfoque se traslada hacia la robustez del sistema al añadir pruebas de integración que verifiquen la comunicación entre módulos. Es fundamental implementar quality gates (umbrales de calidad) que bloqueen código defectuoso y configurar herramientas de análisis estático para monitorear la salud de la arquitectura, garantizando que el crecimiento del software sea ordenado y cumpla con los estándares definidos.
- Finalmente, en la Fase 3: Optimización (Semanas 5-6), se busca alcanzar la madurez operativa al optimizar el performance de los tests para cumplir con los tiempos target. En esta etapa se deben implementar pruebas E2E selectas para los flujos más críticos de usuario y establecer métricas y dashboards en tiempo real que permitan a los stakeholders visualizar el ROI y la estabilidad del producto, cerrando así el ciclo de mejora continua.

9. Conclusiones

- El enfoque en la Detección Temprana y el Feedback Rápido garantiza que el 95% de los defectos sean detectados antes de llegar a producción, optimizando

el ciclo de retroalimentación a menos de 15 minutos. Esta eficiencia técnica no solo acelera el desarrollo, sino que fortalece la Confiabilidad del sistema, logrando una reducción del 80% en los incidentes post-deployment, lo que asegura un entorno de ejecución extremadamente estable.

- Desde una perspectiva de Organizacionales y Predictibilidad, el uso de métricas históricas permite realizar un Planning más preciso, eliminando la incertidumbre en los tiempos de entrega. Al establecer un Framework de Calidad Sostenible, el proyecto queda preparado para un crecimiento escalable, permitiendo que la arquitectura soporte nuevas funcionalidades sin comprometer la integridad de lo que ya está en funcionamiento.
- Finalmente, la implementación de este modelo asegura un ROI Comprobable, donde la ratio costo-beneficio es medible y optimizable en cada etapa del proceso. Al invertir en pruebas unitarias e integración de bajo costo y alto impacto, la organización maximiza la rentabilidad del talento técnico, transformando el aseguramiento de calidad en una ventaja competitiva de alto valor estratégico.